Progetto finale e valutazione

Corso di Frontend Development – Modulo 09

Federico Giuggioloni

ITS – Corso di Frontend Development

Anno formativo 2026/2027

Contenuto del modulo

1. Progetto finale e valutazione
1.1 Il progetto
1.2 Valutazione
1.3 Risorse

Sommario del modulo

1. Progetto finale e valutazione
1.1 Il progetto
1.2 Valutazione
1.3 Risorse

Il progetto finale: tre percorsi

A **minicommerce completa**: tutte le forme viste (web, desktop, PWA, Capacitor, sito vetrina), rifinite e pubblicate, più **due funzionalità nuove** a scelta.

B **Un'app vostra** sulla vostra API dell'anno scorso, in **almeno tre forme**: web + (desktop o mobile) + sito vetrina. Stessi requisiti di qualità di minicommerce.

C **Nativo con agente**: minicommerce (o la vostra app) in versione web come riferimento + **due piattaforme native** realizzate guidando un agente, con specifica, checklist di parità e diario.

In gruppo o da soli

Gruppi di 2–3 con ruoli dichiarati (chi ha fatto cosa deve risultare dai commit). Da soli: percorso A o B ridotti a due forme.

Percorso A: due funzionalità nuove fra queste sei

Carrello, ordine dimostrativo e registrazione ci sono già: queste sei no. Ognuna richiede di estendere **anche l'API**, e progettare le rotte e le tabelle che mancano fa parte del lavoro.

- **Storico degli ordini**: `orders` e `order_items` si riempiono già, ma non c'è nessuna `GET`; servono le rotte di lettura e la pagina del cliente.
- **Scorte vere**: oggi l'ordine non scala lo `stock`; fatelo scalare e tenete una riga di movimento con motivo e autore.
- **Gestione categorie da interfaccia**: oggi ci sono solo le `GET`; servono `POST/PUT/DELETE /api/categories` e il caso "categoria piena".
- **Paginazione del catalogo**: limite e scostamento sull'API, totale nella risposta, pagina nell'URL.
- **Ricerca per codice a barre**: campo `barcode` sugli articoli più il lettore sul mobile (modulo 3).
- **Report delle vendite**: aggregazioni su `order_items` per categoria e periodo, esportabili in CSV.

Recensioni, lista desideri o pagamento vero in sandbox: vanno bene, concordateli con me.

Requisiti minimi (per tutti i percorsi)

- Login e sessione persistente, con ruolo: area admin protetta
- CRUD completo sugli **articoli**, lettura di **categorie** e **utenti**
- Filtri, ordinamento e paginazione **nell'URL** (la paginazione va aggiunta all'API)
- I 4 stati in ogni schermata
- Form validati (client e server)
- Immagini caricate come file, non in base64
- Catalogo leggibile offline, usabile da telefono
- Test: unit + componenti + 1 E2E
- Accessibilità: zero violazioni axe, uso da tastiera
- CI verde su `main`, deploy automatico
- Nessun segreto nel repo, CSP attiva
- Diario dell'uso degli agenti

Cosa consegnare

1. **Repository** pubblico o condiviso con il docente, con storia dei commit leggibile.
2. **README**: cosa fa l'app, architettura (un diagramma), come avviarla, come testarla, URL pubblici.
3. **Video demo** di 5 minuti: le forme dell'app in azione (non le slide).
4. **Relazione** di 3–5 pagine: scelte tecniche e perché, cosa non ha funzionato, diario degli agenti, cosa fareste diversamente.
5. **Presentazione** di 10 minuti + 10 di domande, davanti alla classe.

Griglia di valutazione

Criterio	Peso	Cosa guardo
Funzionalità	25	requisiti coperti, robustezza, casi limite
Qualità del codice	20	struttura, tipi, riuso, leggibilità, confini tra piattaforme
Multi-piattaforma	15	quante forme, quanto codice condiviso, quanto "nativo" il risultato
UX e accessibilità	15	stati, errori comprensibili, tastiera, screen reader, mobile
Qualità di processo	15	test significativi, CI, deploy, sicurezza, changelog
Comprensione	10	risposte alle domande, capacità di spiegare ogni parte

- L'uso degli agenti **non** penalizza né premia di per sé: conta il risultato e la vostra comprensione.
- Codice che non sapete spiegare = punti in meno su "Comprensione" e "Qualità del codice".

Tempi e tappe

Settimana	Traguardo
1	Scelta del percorso, specifica di una pagina, monorepo avviato, CI verde su “hello world”
2	Web app funzionante su articoli, categorie e utenti; test dei flussi base
3	Seconda e terza forma (desktop/mobile/sito); accessibilità e prestazioni
4	Rifiniture, deploy pubblico, video, relazione, prova della presentazione

- Revisione intermedia a fine settimana 2: 15 minuti per gruppo, feedback scritto.
- Chi arriva alla settimana 4 senza deploy funzionante presenta comunque: la rubrica premia ciò che c’è.

All'orale: il codice che avete in mano

A queste domande si risponde solo avendo letto davvero minicommerce.

- Percorrete un **filtro** dal click sulla categoria alla risposta dell'API: dove finisce il valore scelto, quale parametro parte, quale `where` lo applica, cosa torna indietro.
- Cosa succede se l'API risponde **500** mentre si salva un articolo? Chi produce quel 500, chi lo intercetta, cosa vede l'utente, cosa resta a schermo e cosa resta nel database.
- Perché la risposta su un articolo contiene sia `categoryId` sia `category`, e perché in scrittura se ne usa uno solo?
- Il carrello vive **sul server**, in `cart_items`, legato all'utente: perché un amministratore prende **403** su `GET /api/cart`, e cosa succede se aggiungete due volte lo stesso articolo?

All'orale: ordine finto, dati, sessione, immagini

- `POST /api/orders` risponde `accepted` e svuota il carrello: che cosa **non** fa? E del numero di carta digitato nel form, che cosa arriva davvero al server, e quale riga lo taglia?
- Cosa comporta `ON DELETE RESTRICT` quando provate a cancellare una categoria piena di articoli? Chi vi ferma, con quale errore, e come lo spieghereste all'utente.
- Dove finisce il **token** dopo il login, chi lo rimette nelle richieste, e cosa accade esattamente **dopo otto ore** di sessione aperta.
- Perché l'immagine salvata come **data URL base64** è un problema? Misuratelo: peso di una `GET /api/products`, tempo di risposta, memoria sul telefono.
- Caricate un'immagine da 2 MB sulla versione originale: che errore arriva, e da quale pezzo di codice?

All'orale: scelte, qualità, agenti

- Perché avete scelto Tauri e non Electron (o viceversa)? Cosa cambiereste con 10.000 articoli in catalogo?
- Mostratemi un test che fallirebbe se rimettessimo uno dei difetti corretti: il filtro fuori dall'URL, o la categoria piena cancellata senza errore.
- Come fa uno screen reader a leggere questa schermata? E i pulsanti con la sola icona?
- Quale parte di questo codice l'ha scritta l'agente? Cosa gli avete fatto correggere?
- Se domani dovessi fare la versione iOS nativa, da dove partireste?

Domande di verifica

Scegliete una domanda a testa e rispondete a voce al compagno di banco, codice alla mano: se servono più di due minuti, quel pezzo non è ancora vostro.

Risorse per continuare

Documentazione

- https://react.dev – https://reactrouter.com
- https://tailwindcss.com/docs – https://ui.shadcn.com
- https://tauri.app – https://www.electronjs.org
- https://capacitorjs.com – https://vite-pwa-org.netlify.app
- https://astro.build – https://nextjs.org
- https://orm.drizzle.team – https://expressjs.com
- https://developer.apple.com/swiftui – https://developer.android.com/compose

Qualità

- https://web.dev (prestazioni, PWA, vitals)
- https://www.w3.org/WAI/WCAG22/quickref
- https://testing-library.com – https://playwright.dev
- https://cheatsheetseries.owasp.org
- https://developer.mozilla.org (sempre)

Agenti

- Documentazione ufficiale dello strumento scelto; changelog mensili: cambiano in fretta.